

‘K’- Scheme Summer Semester (2025-2026)

UT-2 Question Bank for MIC

Unit-III

Q1. Attempt any THREE.

(3x2=6 Marks)

1. State the use of STC and CMC instruction.

Ans: STC: This instruction is used to set the carry flag. When this instruction is executed the carry flag is set to 1

CMC: This instruction is used to complement the value of the carry flag. When this instruction is executed the value in the carry flag gets inverted.

2. State the function of XLAT instruction.

Ans: The XLAT (Translate) instruction in Intel 8086 assembly language is used for table lookup / byte translation. It replaces the value in the AL register with a value taken from a lookup table in memory.

3. Compare MUL and IMUL instructions of 8086. (Two points)

Ans: Difference between MUL and IMUL Instruction

SrNo	MUL	IMUL
(i)	MUL instruction is used for multiplication of Unsigned numbers	IMUL instruction is used for multiplication of Signed Numbers
(ii)	Assumes the number is positive and does not consider sign bit	Works with both positive and negative numbers and considers sign bit

4. Differentiate between ROL and RCL instruction (any two points)

Ans: Difference between ROL and RCL Instruction

SrNo	ROL	RCL
(i)	It is used to rotate a byte or a word to the left	It is used to rotate a byte or a word to the left through carry flag
(ii)	Syntax: ROL Destination, Count	Syntax: RCL Destination, Count

Q2. Attempt any THREE

(4x3= 12 Marks)

1. Explain with suitable example the instructions given below:

(i) DAA (ii) AAM

Ans: (i) DAA:

- It stand for Decimal Adjustment After Addition
- It is used to adjust the decimal after the addition operation

- It makes the result in Packed BCD form after the BCD addition is performed
- It works only on AL register

Flags Affected: All flags are updated after this instruction. OF becomes undefined

Example:

For AL register ONLY

If $D3 - D0 > 9$ OR Auxiliary Carry Flag is Set, ADD 06H to AL.

If $D7 - D4 > 9$ OR Carry Flag is Set, ADD 60 H to AL.

Assume : AL = 14H,

CL = 28H

Then, ADD AL,CL gives

AL = 3CH

Now, DAA gives

AL = 42(06 is added to AL as $C > 9$)

2. Explain any four string instructions of 8086 with example

Ans:

1] REP:

REP is a prefix which is written before one of the string instructions. It will cause the length counter CX to be decremented and the string instruction to be repeated until CX becomes 0.

Two more prefix.

REPE/REPZ: Repeat if Equal /Repeat if Zero.

It will cause string instructions to be repeated as long as the compared bytes or words are equal and $CX \neq 0$.

REPNE/REPNZ: Repeat if not equal/Repeat if not zero.

It repeats the strings instructions as long as compared bytes or words are not equal and $CX \neq 0$.

Example: REP MOVSB

2] MOVS/ MOVSB/ MOVSW - Move String byte or word.

Syntax:

MOVS destination, source

MOVSB destination, source

MOVSW destination, source

Operation: $ES:[DI] \leftarrow DS:[SI]$

It copies a byte or word from a location in data segment to a location in extra segment. The offset of source is pointed by SI and offset of destination is pointed by DI. CX register contains counter and direction flag (DF) will be set or reset to auto increment or auto decrement pointers after one move.

Example

LEA SI, Source

LEA DI, destination

CLD

MOV CX, 04H

REP MOVSB

3] CMPS /CMPSB/CMPSW: Compare string byte or Words.

Syntax:

CMPS destination, source

CMPSB destination, source

CMPSW destination, source

Operation: Flags affected < ----- DS:[SI]- ES:[DI]

It compares a byte or word in one string with a byte or word in another string. SI holds the offset of source and DI holds the offset of destination strings. CS contains counter and DF=0 or 1 to auto increment or auto decrement pointer after comparing one byte/word.

Example

LEA SI, Source

LEA DI, destination

CLD

MOV CX, 100

REPE CMPSB

4] SCAS/SCASB/SCASW: Scan a string byte or word.

Syntax: SCAS/SCASB/SCASW

Operation: Flags affected < ----- AL/AX-ES: [DI]

It compares a byte or word in AL/AX with a byte /word pointed by ES: DI. The string to be scanned must be in the extra segment and pointed by DI. CX contains counter and DF may be 0 or 1. When the match is found in the string execution stops and ZF=1 otherwise ZF=0.

Example

```
LEA DI, destination
MOV AL, 0DH
MOV CX, 80H
CLD
REPNE SCASB
```

5] LODS/LODSB/LODSW: Load String byte into AL or Load String word into AX.

Syntax: LODS/LODSB/LODSW

Operation: AL/AX < ----- DS: [SI]

IT copies a byte or word from string pointed by SI in data segment into AL or AX. CX may contain the counter and DF may be either 0 or 1

Example

```
LEA SI, destination
CLD
LODSB
```

6] STOS/STOSB/STOSW (Store Byte or Word in AL/AX)

Syntax: STOS/STOSB/STOSW

Operation: ES:[DI] < ----- AL/AX

It copies a byte or word from AL or AX to a memory location pointed by DI in extra segment CX may contain the counter and DF may either set or reset

Example:

```
LEA SI, destination
CLD
STOSB
```

Q3. Attempt any TWO**(6x2= 12 Marks)****1. Write an instruction to perform following operations:****(i) Multiply BL by 88H**

```
MOV AL,88H
```

```
MUL BL
```

(ii) Signed division of AL by BL

IDIV BL

(iii) Move 4000H to DS register

MOV AX, 4000H

MOV DS,AX

(iv) Rotate content of AX register to left 4 times.

MOV CL,04H

ROL AX,CL

OR

MOV CL,04H

RCL AX,CL

(v) Shift the content of BX register to right 3 times.

MOV CL,03H

SHL BX,CL

(vi) Load SS with FF00H

MOV AX,FF00H

MOV SS,AX

(vii) Load 4000H in SP

MOV SP,4000H

(viii) Rotate AX register to right 3 times through carry

MOV CL,03H

RCL AX,CL

(ix) Add 100H to the contents of AX register

ADD AX,0100H

(x) Copy the contents of BX register to CS

The content of CS register cannot be modified or accessed directly, Hence no instruction can be performed for given statement

2. Explain with example any three Shift and any three Rotate instructions

Ans:

Shift Instructions:

1. SAL/SHL Destination,Count:

- This instruction is used to shift the data bits to the left.
- The MSB is shifted into carry flag and LSB gets 0.
- The bits are shifted count no. of times.

- If the count = 1, it can be directly specified in instruction, if count > 1 CL register has to be loaded with count.

Flags Affected: CF contains the bit most recently shifted out from MSB. Other flags affected are: OF, SF, ZF, PF.

Eg. SHL AX,01 OR SAL AX,01

Let AX = 0001001000110100

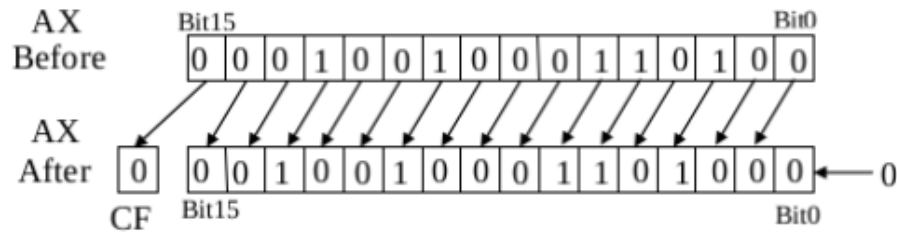


Fig: A 16-bit value form AX Register is Shifted Left once (SHL AX,1)

2. SHR Destination,Count:

- This instruction is used to shift the data bits to the right.
- The LSB is shifted into carry flag and MSB gets 0.
- The bits are shifted count no. of times.
- If the count = 1, it can be directly specified in instruction, if count > 1 CL register has to be loaded with count.

Flags Affected: CF contains the bit most recently shifted out from MSB. Other flags affected are: OF, SF, ZF, PF.

Eg. SHR AX,1

Let's consider the value of AX = 0001001000110100

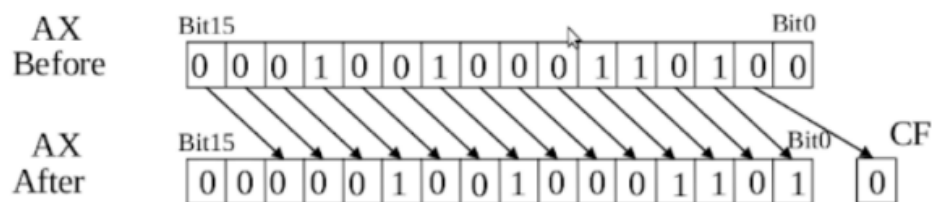


Fig: A 16-bit value from AX Register is Shifted Right once (SHR AX,1)

3. SAR Instruction:

- This instruction is used to shift the data bits to the right
- As a bit is shifted out of the MSB position, a copy of the old MSB is put in the MSB position i.e., the sign bit is copied into the MSB

- The LSB is shifted into carry flag
- The bits shifted count no of times and if the count = 1, it can be directly specified in instruction, if count > 1, CL register has to be loaded with count

Flags Affected: CF contains the bit most recently shifted in from LSB. For a count of one OF will be 1, after multi bit it will be 0. Other flags affected: SF, ZF, PF, AF

Eg. SAR AX,1

Let's consider the value of AX = 000100100011010

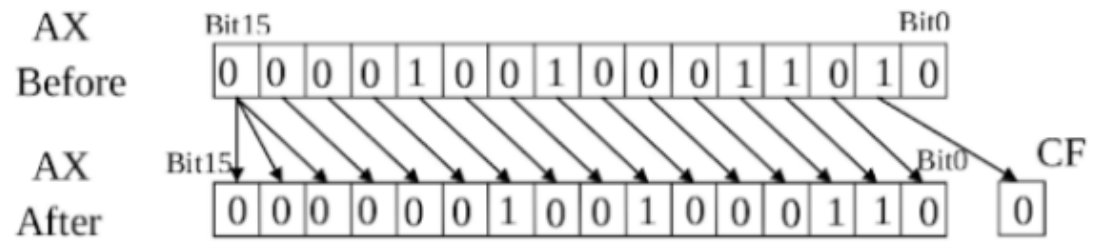


Fig: A 16-bit value from AX Register is Shifted Right once (SAR AX,1)

Rotate Instructions:

1.ROL Destination,Count:

- This instruction rotates the data bits in a register from MSB to LSB
- When ROL is executed, all bits shift towards left and the MSB bit gets shifted out and rotates back to LSB bit of the same register as well as a copy is sent to the carry flag.

Flags Affected: ROL affects CF and OF

Eg. ROL AL,1

Let's consider the value of AL = 01010111

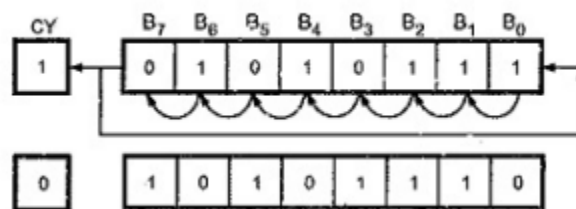


Fig: ROL Instruction

2.ROR Destination,Count

- This instruction rotates the data bits in a register from LSB to MSB.
- When ROR is executed, all bits shift towards right and the LSB bit gets shifted out and rotates back to MSB bit of the same register as well as a copy is sent to the carry flag.

Flags Affected: ROR affects CF and OF

Eg. ROR AL,1

Let's consider value in BL = 10100011

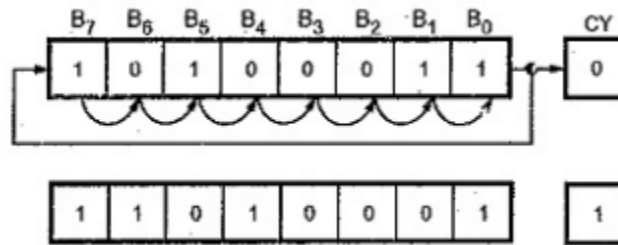


Fig: ROR Instruction

3.RCL Destination,Count

- This instruction rotates the data bits in a register from MSB to carry flag to LSB.
- When RCL is executed, all the bits shift towards left and the MSB bit gets shifted out and goes to carry flag, the bit which previously stored in the carry flag is placed in the LSB bit of the same register.

Flags Affected: RCL affects CF and OF

Eg. RCL AL,1

Let's consider the value of BL = 10101101

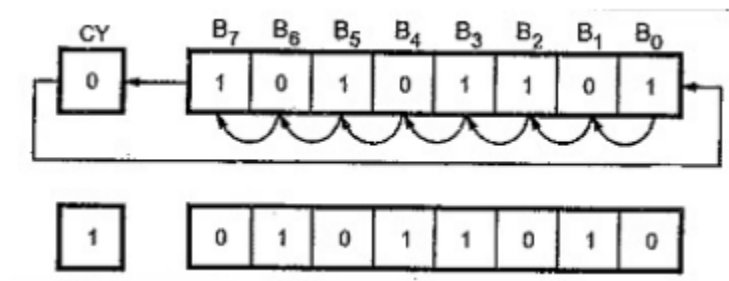


Fig: RCL Instruction

4.RCR Destination,Count

- This instruction rotates the data bits in a register from LSB to carry flag to MSB.
- When RCR is executed, all the bits shift towards right and the LSB bit gets shifted out and goes to carry flag, the bit which previously stored in the carry flag is placed in the MSB bit of the same register.

Flags Affected: RCR affects CF and OF

Eg. RCR BL,1

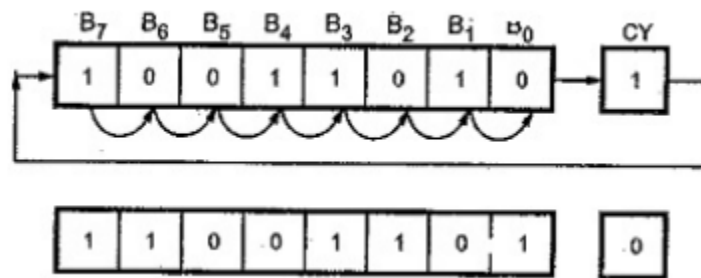


Fig: RCR Instruction

Unit-IV

Q1. Attempt any THREE.

(3x2=6 Marks)

1. Write an ALP to find length of string

Ans:

assume cs:code

assume ds:data

data segment

str1 db "Good Morning \$"

len db ?

data ends

code segment

start: MOV ax,data

MOV ds,ax

MOV ax,0000h

LEA si,str1

UP: MOV al,[si]

CMP al,"\$"

JE DOWN

INC SI

INC DL

JMP UP

DOWN: MOV len,dl

INT 03h

code ends

end start

2. Write an ALP to find if the given number is negative or positive

Ans:

```
assume cs:code
assume ds:data
data segment
num DB 10H
msg1 DB "Number is positive $"
msg2 DB "Number is negative $"
data ends
code segment
start:  MOV AX,data
        MOV DS,AX
        MOV AX,0000H
        MOV AL,num
        CMP AL,00H
        JS down
        LEA DX, msg1
        MOV AH,09H
        INT 21H
        JMP exit
down:   LEA DX,msg2
        MOV AH,09H
        INT 21H
exit:   MOV AH,4CH
        INT 21H
code ends
end start
```

3. Write an ALP to add/sub two 8-bit numbers

Ans:

```
assume cs:code
assume ds:data
data segment
num1 DB 10H
```

```

num2 DB 20H
res DB ?
data ends
code segment
start:  MOV AX,data
        MOV DS,AX
        MOV AX,0000H
        MOV AL,num1
        MOV BL,num2
        ADD AL,BL
        MOV res,AL
        INT 03H
code ends
end start

```

AND

```

assume cs:code
assume ds:data
data segment
num1 DB 10H
num2 DB 20H
res DB ?
data ends
code segment
start:  MOV AX,data
        MOV DS,AX
        MOV AX,0000H
        MOV AL,num1
        MOV BL,num2
        SUB AL,BL
        MOV res,AL
        INT 03H

```

code ends

end start

Q2. Attempt any THREE

(4x3= 12 Marks)

1. Describe the Programming Model of 8086

Ans: Programming Model of 8086

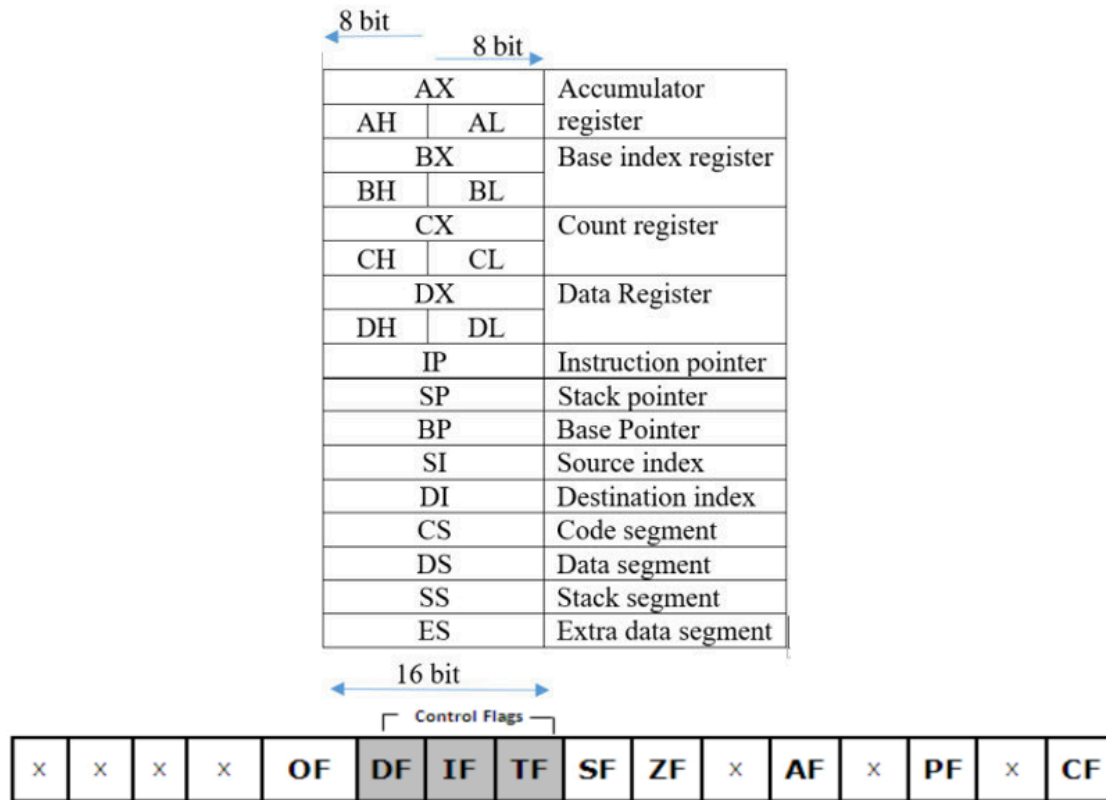


Fig: Programming Model of 8086

2. Write an ALP to perform block transfer operation of 10 numbers.

Ans:

assume cs:code

assume ds:data

data segment

arr1 DB 05H,10H,15H,20H,25H,30H,35H,40H,45H,50H

arr2 10 DUP (?)

data ends

code segment

start: MOV AX,data

MOV DS,AX

```

        MOV AX,0000H
        LEA SI,arr1
        LEA DI,arr2
        MOV CL,0AH
up:     MOV AL,[SI]
        MOV [DI],AL
        INC SI
        INC DI
        LOOP up
        INT 03H

```

code ends

end start

3. Write an ALP to find the smallest number

Ans:

assume cs:code

assume ds:data

data segment

arr DB 10H,07H,15H,27H,02H

smallest DB ?

data ends

code segment

start: MOV AX,data

MOV DS,AX

MOV AX,0000H

LEA SI,arr

MOV CL,04H

MOV AL,[SI]

up: INC SI

MOV BL,[SI]

CMP AL,BL

JS down

MOV AL,BL

down: LOOP up

MOV smallest,AL

INT 03H

code ends

end start

4. Write an ALP to arrange array in ascending order (assume 5 numbers)

Ans:

assume cs : code

assume ds : data

data segment

array db 10h,15h,9h,7h,4h

data ends

code segment

start: mov ax,data

mov ds,ax

mov ax,0000h

mov cl,04

up2: mov ch,04

lea si,array

up1: mov al,[si]

mov bl,[si+1]

cmp al,bl

jns down

mov dl,[si+1]

xchg dl,[si]

mov [si+1],dl

down: inc si

dec ch

jnz up1

dec cl

jnz up2

int 03H

code ends

end start

5. Write an ALP to count the number of '0' in an 8-bit number.

Ans:

```
assume cs:code
assume ds:data
data segment
number db 10011011b
zeroes dw ?
data ends
code segment
start:  mov ax,data
        mov ds,ax
        mov ax,0000h
        mov al,number
        mov cl,08
up:     ror al,1
        jc one
        inc di
        dec cl
        jnz up
        jmp exit
one:    dec cl
        jnz up
        mov zeroes, di
exit:   int 03H
code ends
end start
```

6. Write an ALP to reverse a string.

Ans:

```
assume cs : code
assume ds : data
data segment
string db "MAHARASHTRA$"
strrev db 20 dup (?)
```

```
msg db "The reverse of the string is: $"
```

```
data ends
```

```
code segment
```

```
start:  mov ax,data
```

```
        mov ds,ax
```

```
        mov ax,0000h
```

```
        lea si,string
```

```
up:     mov al,[si]
```

```
        cmp al,'$'
```

```
        je down
```

```
        inc si
```

```
        jmp up
```

```
down:   lea di,strrev
```

```
        mov cx,si
```

```
up1:    dec si
```

```
        mov al,[si]
```

```
        mov [di],al
```

```
        inc di
```

```
        dec cx
```

```
        jnz up1
```

```
        mov al,'$'
```

```
        mov [di],al
```

```
        lea dx,msg
```

```
        mov ah,09
```

```
        int 21h
```

```
        lea dx,strt
```

```
        mov ah,09
```

```
        int 21h
```

```
        mov ah,4ch
```

```
        int 21h
```

```
code ends
```

```
end start
```

Q3. Attempt any TWO**(6x2= 12 Marks)**

- 1. Write an ALP for concatenation of two strings. Draw flowchart and assume suitable data.**

Ans: (Refer Manual for Flowchart)

assume cs : code

assume ds : data

data segment

msg1 db "The two strings are :\$"

string1 db 10,13, "If you don't study \$"

string2 db 10,13, "You will fail in MIC ☺ \$"

msg2 db 10,13, "After concatenation the strings are as follows :\$"

data ends

code segment

start: mov ax,data

mov ds,ax

mov ax,0000h

lea dx,msg1

mov ah,09

int 21h

lea dx,string1

mov ah,09

int 21h

lea dx,string2

mov ah,09

int 21h

lea si,string1

lea di,string2

up: inc si

mov al,[si]

cmp al,'\$'

jne up

up1: mov al,[di]

mov [si],al

```

inc si
inc di
cmp al,'$'
jne up1
lea dx,msg2
mov ah,09
int 21h
lea dx,string1
mov ah,09
int 21h
mov ah,4ch
int 21h
code ends
end start

```

2. Write an ALP to arrange numbers in array in descending order (assume 10 numbers). Also Draw the flowchart

Ans: (Refer Manual for Flowchart)

```

assume cs : code
assume ds : data
data segment
array db 10h,15h,09h,07h,04h,12h,15h,27h,14h,06h
data ends
code segment
start:  mov ax,data
        mov ds,ax
        mov ax,0000h
        mov cl,09
up2:    mov ch,09
        lea si,array
up1:    mov al,[si]
        mov bl,[si+1]
        cmp al,bl
        js down

```

```

    mov dl,[si+1]
    xchg dl,[si]
    mov [si+1],dl
down: inc si
    dec ch
    jnz up1
    dec cl
    jnz up2
    int 03H
code ends
end start

```

Unit-V

Q1. Attempt any THREE.

(3x2=6 Marks)

1. Define MACRO and Procedure. Write its syntax

Ans: MACRO: A MACRO is a group of small instructions that usually performs one task. It is a reusable section of a software program. A macro can be defined anywhere in a program using directive MACRO& ENDM.

Syntax: Macro_name MACRO Parameter

```

_____
_____ Body of the MACRO
_____
ENDM Macro_name

```

PROCEDURE: A procedure is a group of instructions that usually performs one task. It is a reusable section of a software program which is stored in memory once but can be used as often as necessary. A procedure can be of two types. 1) Near Procedure 2) Far Procedure

Syntax: Procedure_name PROC NEAR/FAR

```

_____
_____ Body of the Procedure
_____
RET
ENDP Procedure_name

```

2. How's the procedure called from the main program

Ans: The procedure is called using CALL instruction. When the CALL instruction is invoked the program control is transferred to the subprogram or subroutine. CALL can be NEAR or FAR depending on the code segment in which the procedure is present.

3. Compare Inter-Segment CALL and Intra-Segment CALL

Ans: (Note: It is can also be asked as NEAR CALL or FAR Call)

SrNo.	Intra-Segment CALL	Inter-Segment CALL
1	If the location of procedure in which the control is to be transferred lies in the same code segment from which it is called then the call is called intra-segment call	If the location of the procedure in which the control of program is to be transferred lies in the different code segment than from which it is called then the call is called inter-segment call
2	It is also called as NEAR CALL	It is also called as FAR CALL
3	The old IP value is pushed on to the stack and IP: gets assigned to the procedure	It replaces the old value of CS:IP pairs with new CS:IP
4	Value of IP is pushed onto the stack	Value of both CS:IP is pushed onto the stack
5	Less stack location is required	More stack location is required
6	It used keyword NEAR	It used keyword FAR
7	Example: CALL Delay NEAR	Example: CALL Delay FAR

Q2. Attempt any THREE

(4x3= 12 Marks)

1. Difference between PROCEDURE and MACRO

Ans:

Procedure	Macro
Procedures are used for large group of instructions to be repeated	Procedures are used for small group of instructions to be repeated.
Object code is generated only once in memory.	Object code is generated every time the macro is called.
CALL & RET instructions are used to call procedure and return from procedure.	Macro can be called just by writing its name.
Length of the object file is less	Object file becomes lengthy.
Directives PROC & ENDP are used for defining procedure.	MACRO and ENDM are used for defining MACRO
Directives More time is required for its execution	Less time is required for it's execution
Procedure can be defined as <pre> Procedure_name PROC ---- ----- Procedure_name ENDP </pre>	Macro can be defined as <pre> MACRO-name MACRO [ARGUMENT,..... ARGUMENT N] ----- ----- ENDM </pre>
For Example <pre> Addition PROC near ----- Addition ENDP </pre>	For Example <pre> Display MACRO msg ----- ENDM </pre>

2. Describe Recursive and Reentrant procedure with diagram.

Ans: Recursive Procedure: A recursive procedure is a procedure which calls itself. This results in the procedure call to be generated from within the procedures again and again.

The recursive procedures keep on executing until the termination condition is reached.

The recursive procedures are very effective to use and to implement but they take a large amount of stack space and the linking of the procedure within the procedure takes more time as well as puts extra load on the processor.

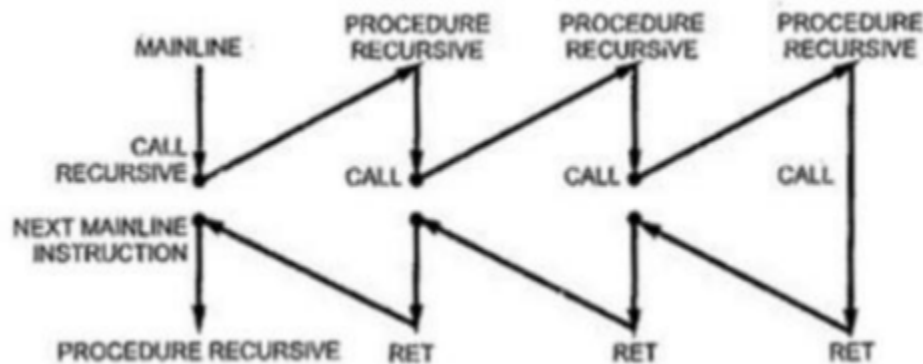


Fig: Recursive Procedure

Reentrant Procedure: In some situations it may happen that Procedure 1 is called from the main program, Procedure2 is called from Procedure1 and Procedure1 is again called from Procedure2. In this situation program execution flow re enters in the procedure1. These types of procedures are called Reentrant Procedures.

A procedure is said to be re-entrant, if it can be interrupted, used and re-entered without losing or writing over anything.

Like recursive procedure re-entrant procedure also needs termination condition or else it can go to infinite procedure calls and lead to machine halt

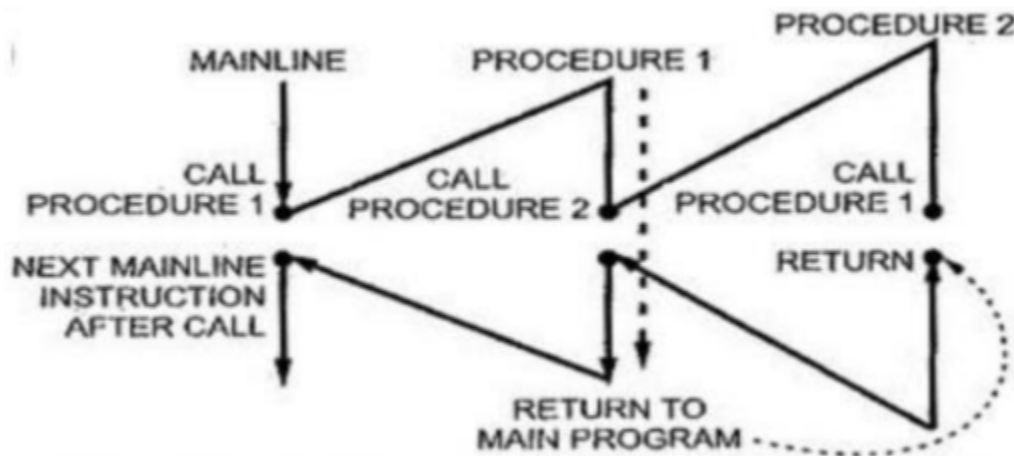


Fig: Reentrant Procedure

- Write an assembly language program to solve $P = X^2 + Y^2$ using procedure. (X and Y are 8-bit numbers)

Ans:

```
assume cs:code
```

```
assume ds:data
```

```
data segment
```

```
X DB 10H
```

Y DB 20H

P DW ?

data ends

code segment

start: MOV AX,data

MOV DS,AX

MOV AX,0000H

CALL Sqr1

CALL Sqr2

CALL Adding

INT 03h

Sqr1 PROC

MOV AL,X

MUL AL

MOV CX,AX

RET

ENDP Sqr1

Sqr2 PROC

MOV AL,Y

MUL AL

RET

ENDP Sqr2

Adding PROC

ADD AX,CX

MOV Z,AX

RET

ENDP Adding

code ends

end start

4. Write an ALP for $Z = (P+Q) \times (R+S)$ using PROCEDURE

Ans:

assume cs:code

assume ds:data

data segment

a db 10h

b db 20h

c db 30h

d db 40h

temp1 db ?

temp2 db ?

z dw ?

data ends

code segment

start: mov ax,data

mov ds,ax

mov ax,0000h

CALL ADD1

CALL ADD2

CALL MULTIPLY

INT 03h

PROC ADD1

mov al,a

mov bl,b

add al,bl

mov temp1,al

RET

ENDP ADD1

PROC ADD2

```
        mov al,c
        mov bl,d
        add al,bl
        mov temp2,al
RET
ENDP ADD2
```

```
PROC MULTIPLY
        mov al,temp1
        mov bl,temp2
        mul bl
        mov z,ax
RET
ENDP MULTIPLY
code ends
end start
```

5. Write an ALP to find factorial of a given number

Ans:

```
assume cs:code
assume ds:data
data segment
num DB 04H
res DW ?
data ends
code segment
start:  mov ax,data
        mov ds,ax
        mov ax,0000h
        call fact
        int 03h
```

```
fact PROC
```

```

        cmp ax,01h
        jz exit
        push ax
        dec ax
        call fact
        pop ax
        mul res
        mov res,ax
        ret
exit:    mov res,01h
        ret
        ENDP fact
code ends
end start

```

Q3. Attempt any TWO

(6x2= 12 Marks)

1. Write an ALP for $Z = (P+Q) \times (R+S)$ using MACRO. Draw flow chart of the same

Ans: (For flowchart refer manual)

assume cs:code

assume ds:data

data segment

p db 10h

q db 20h

r db 30h

s db 40h

temp1 db ?

temp2 db ?

z dw ?

data ends

addition MACRO a,b,ans

mov al,a

mov bl,b

add al,bl

```
        mov ans,al
ENDM addition
```

```
multiply MACRO x,y,res
        mov al,a
        mov bl,b
        mul bl
        mov res,ax
ENDM multiply
```

```
code segment
start:  mov ax,data
        mov ds,ax
        mov ax,0000h
        addition p,q,temp1
        addition r,s,temp2
        multiply temp1,temp2,z
        int 03h
code ends
end start
```